Aaditya Dawar

1024150093

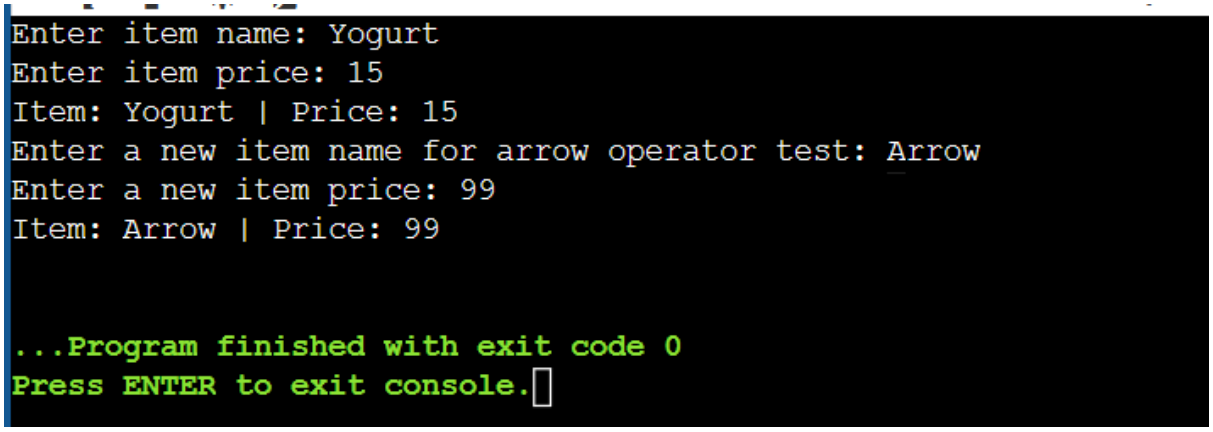
2O13

Lab Assignment 3

1. Write a program to implement (a) pointer to an object (b) this pointer. Practice both '.' (dot operator) and '→' (arrow operator).

```
#include <iostream>
#include <string>

class Item {
private:
    std::string name;
    double price;

public:
    void setData(std::string name, double price) {
        this->name = name;
        this->price = price;
    }

    void display() {
        std::cout << "Item: " << this->name << " | Price: " << this->price << std::endl;
    }
};

int main() {
    Item obj;
    Item* ptr = &obj;

    std::string inputName;
    double inputPrice;

    std::cout << "Enter item name: ";
    std::cin >> inputName;
    std::cout << "Enter item price: ";
    std::cin >> inputPrice;

    (*ptr).setData(inputName, inputPrice);
    (*ptr).display();
```

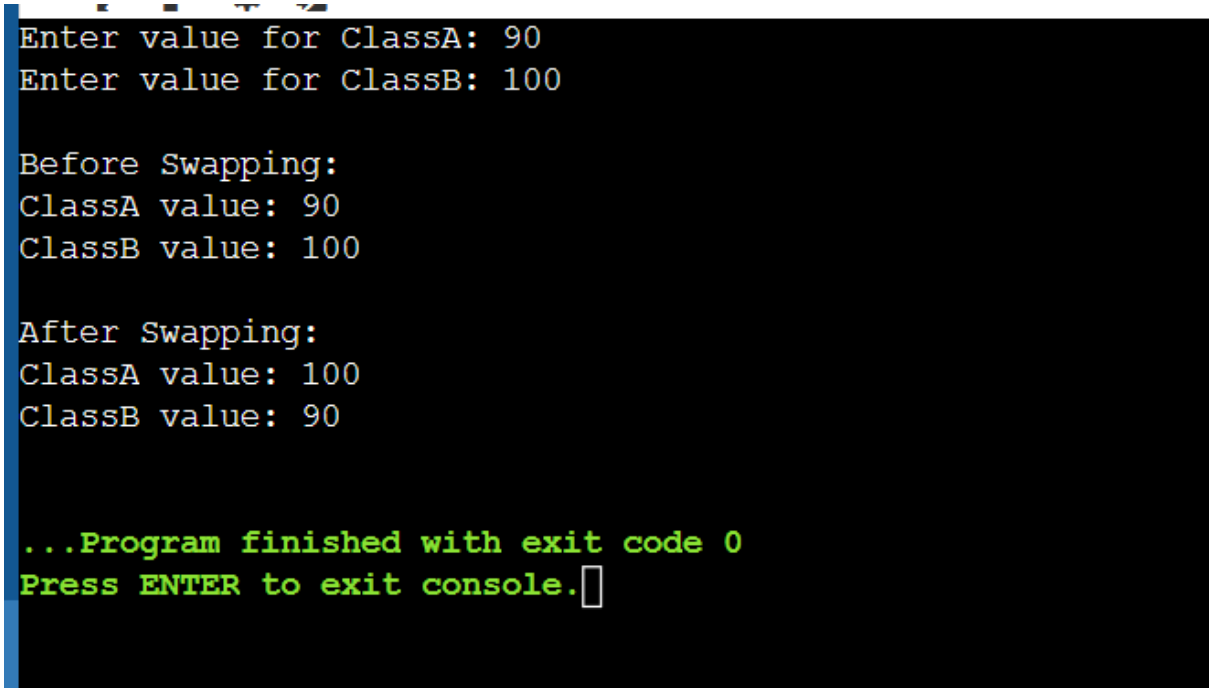
```
    std::cout << "Enter a new item name for arrow operator test: ";
    std::cin >> inputName;
    std::cout << "Enter a new item price: ";
    std::cin >> inputPrice;

    ptr->setData(inputName, inputPrice);
    ptr->display();

    return 0;
}
```



2. Write a program to swap private values of two classes using a friend function.

```
#include <iostream>

class ClassB;

class ClassA {
private:
    int valueA;

public:
    void input() {
        std::cout << "Enter value for ClassA: ";
        std::cin >> valueA;
    }

    void display() {
        std::cout << "ClassA value: " << valueA << std::endl;
    }

    friend void swapValues(ClassA&, ClassB&);
};

class ClassB {
private:
    int valueB;
```

```
public:
    void input() {
        std::cout << "Enter value for ClassB: ";
        std::cin >> valueB;
    }

    void display() {
        std::cout << "ClassB value: " << valueB << std::endl;
    }

    friend void swapValues(ClassA&, ClassB&);
};

void swapValues(ClassA& a, ClassB& b) {
    int temp = a.valueA;
    a.valueA = b.valueB;
    b.valueB = temp;
}

int main() {
    ClassA objA;
    ClassB objB;

    objA.input();
    objB.input();

    std::cout << "\nBefore Swapping:" << std::endl;
    objA.display();
    objB.display();

    swapValues(objA, objB);

    std::cout << "\nAfter Swapping:" << std::endl;
    objA.display();
    objB.display();

    return 0;
}
```



3. Write a program to add data objects of two different classes using friend functions.

```
#include <iostream>

class ClassB;

class ClassA {
private:
    int numA;

public:
    void input() {
        std::cout << "Enter value for ClassA object: ";
        std::cin >> numA;
    }

    friend int add(ClassA, ClassB);
};

class ClassB {
private:
    int numB;

public:
```

```
    void input() {
        std::cout << "Enter value for ClassB object: ";
        std::cin >> numB;
    }

    friend int add(ClassA, ClassB);
};

int add(ClassA a, ClassB b) {
    return a.numA + b.numB;
}

int main() {
    ClassA objA;
    ClassB objB;

    objA.input();
    objB.input();

    int sum = add(objA, objB);

    std::cout << "The sum of data from both classes is: " << sum << std::endl;

    return 0;
}
```

```
Enter value for ClassA object: 90
Enter value for ClassB object: 100
The sum of data from both classes is: 190


...Program finished with exit code 0
Press ENTER to exit console.
```

4. Write a program to demonstrate the working of friend class.

```
#include <iostream>
#include <string>

class Database {
```

```
private:
    std::string secretToken;

public:
    void setToken() {
        std::cout << "Enter the secret token for the database: ";
        std::cin >> secretToken;
    }

    friend class Admin;
};

class Admin {
public:
    void showToken(Database& db) {
        std::cout << "Admin Access - Database Token: " << db.secretToken << std::endl;
    }

    void modifyToken(Database& db) {
        std::string newToken;
        std::cout << "Admin Access - Enter new token: ";
        std::cin >> newToken;
        db.secretToken = newToken;
        std::cout << "Token successfully updated by Admin." << std::endl;
    }
};

int main() {
    Database myDB;
    Admin myAdmin;

    myDB.setToken();

    std::cout << "\nInitial View:" << std::endl;
    myAdmin.showToken(myDB);

    std::cout << "\nModifying Data:" << std::endl;
    myAdmin.modifyToken(myDB);

    std::cout << "\nFinal View:" << std::endl;
    myAdmin.showToken(myDB);

    return 0;
}
```

5. Write a program using Array of Objects to display area of multiple rectangles.

```
#include <iostream>

class Rectangle {
private:
   float width;
   float height;

public:
   void setData() {
      std::cout << "Enter width: ";
      std::cin >> width;
      std::cout << "Enter height: ";
      std::cin >> height;
   }

   float calculateArea() {
      return width * height;
   }

   void display(int index) {
      std::cout << "Rectangle " << index + 1 << " -> Width: " << width
            << ", Height: " << height << ", Area: " << calculateArea() << std::endl;
   }
};

int main() {
   int n;
   std::cout << "Enter the number of rectangles: ";
   std::cin >> n;

   Rectangle rects[n];

   for (int i = 0; i < n; i++) {
      std::cout << "\nInput for Rectangle " << i + 1 << ":" << std::endl;
      rects[i].setData();
   }

   std::cout << "\n--- Displaying Results ---" << std::endl;
   for (int i = 0; i < n; i++) {
      rects[i].display(i);
   }
```

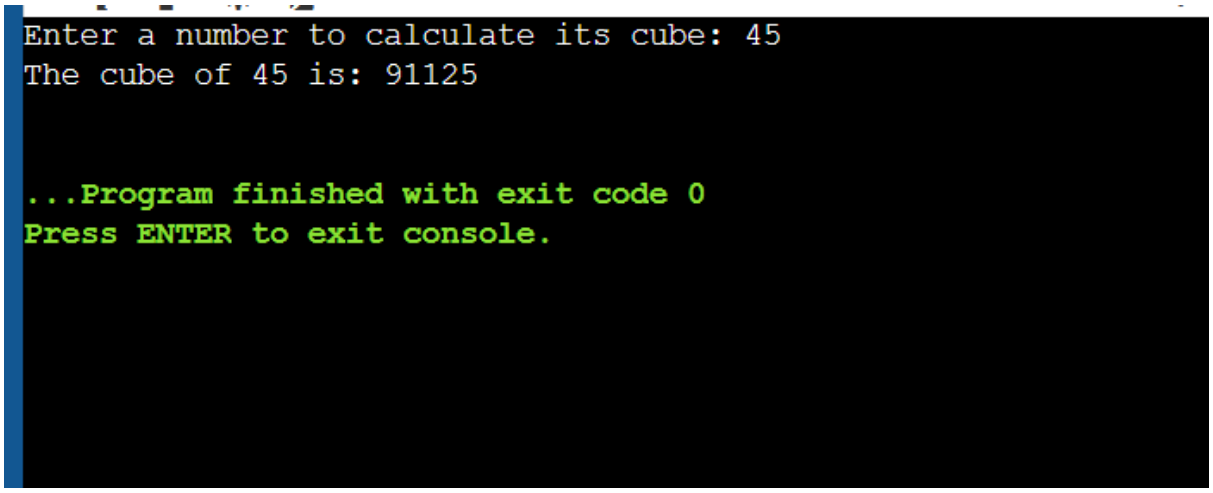
```
    return 0;
}
```

```
Enter the number of rectangles: 2

Input for Rectangle 1:
Enter width: 10
Enter height: 5

Input for Rectangle 2:
Enter width: 200
Enter height: 3

--- Displaying Results ---
Rectangle 1 -> Width: 10, Height: 5, Area: 50
Rectangle 2 -> Width: 200, Height: 3, Area: 600
```

6. Write a program to define function ***cube()*** as inline for calculating cube of a number.

```
#include <iostream>


inline double cube(double num) {
    return num * num * num;
}


int main() {
    double val;


    std::cout << "Enter a number to calculate its cube: ";
    std::cin >> val;


    std::cout << "The cube of " << val << " is: " << cube(val) << std::endl;
```

```
    return 0;
}
```



7. Write a program to pass an object as an argument and return the object from a function.
    a. Use pass-by-value
    b. Use pass-by reference

```
#include <iostream>
#include <string>

class Counter {
public:
    int count;

    void input() {
        std::cout << "Enter count value: ";
        std::cin >> count;
    }

    void display() {
        std::cout << "Count: " << count << std::endl;
    }
};
```

```
Counter incrementByValue(Counter c) {
    c.count += 10;
    std::cout << "Inside incrementByValue (modified copy): " << c.count << std::endl;
    return c;
}

Counter& incrementByReference(Counter& c) {
    c.count += 20;
    std::cout << "Inside incrementByReference (modified original): " << c.count << std::endl;
    return c;
}

int main() {
    Counter obj1, obj2, result;

    std::cout << "--- Pass by Value Test ---" << std::endl;
    obj1.input();
    result = incrementByValue(obj1);
    std::cout << "Original Object after Value Call: ";
    obj1.display();
    std::cout << "Returned Object from Value Call: ";
    result.display();

    std::cout << "\n--- Pass by Reference Test ---" << std::endl;
    obj2.input();
    incrementByReference(obj2);
    std::cout << "Original Object after Reference Call: ";
    obj2.display();
```

```
    return 0;
}
```

```
--- Pass by Value Test ---
Enter count value: 5
Inside incrementByValue (modified copy): 15
Original Object after Value Call: Count: 5
Returned Object from Value Call: Count: 15

--- Pass by Reference Test ---
Enter count value: 3
Inside incrementByReference (modified original): 23
Original Object after Reference Call: Count: 23


...Program finished with exit code 0
Press ENTER to exit console.
```